

Tesp de Inteligência Artificial

Programação de Computadores II

Funções

Funções – para que servem

- Extremamente importantes para simplificar o código, e para poderem ser reutilizadas

Funções – elementos essenciais

□ A sintaxe de definição de uma função:

```
def NomeDaFuncao(argumento1, argumento2, ...):  
    instruções  
    return saída
```

- “def”: define uma função
- NomeDaFuncao: define o nome a atribuir à função
- Argumentos: são os inputs para a função e em Python são opcionais
- Instruções: linhas de código a desenvolver na função
- return: devolve valores calculados

Funções – variáveis locais

□ As variáveis são locais

```
#Funções e variáveis locais

var = 7

def change_value():
    var = 13
    print('Valor da variável dentro da função: ',var)

print('Antes de mudar o valor',var)
change_value()
print('Após mudar: ',var)
```

Antes de mudar o valor 7

Valor da variável dentro da função: 13

Após mudar: 7

Funções – variáveis globais

- Para definir uma variável como global

```
#Funções e variáveis locais
```

```
var = 7
```

```
def change_value():
```

```
    global var
```

```
    var = 13
```

```
    print('Valor da variável dentro da função: ',var)
```

```
print('Antes de mudar o valor',var)
```

```
change_value()
```

```
print('Após mudar: ',var)
```

Antes de mudar o valor 7

Valor da variável dentro da função: 13

Após mudar: 13

Funções – sem argumentos e sem retorno

- Ambas as funções definidas anteriormente são funções que não recebem nem devolvem nada, são exemplo de procedimentos.

Funções – com 1 argumento e sem retorno

- Passagem de um argumento para a função

```
#Função com 1 argumento e sem retorno
```

```
var = 7
```

```
def change_value(var):  
    print('Valor do argumento: ',var)
```

```
change_value(var)
```

```
Valor do argumento: 7
```

Funções – com 2 argumentos e sem retorno

□ Passagem de dois argumentos para a função

```
#Função com 2 argumentos e sem retorno
```

```
var = 7
```

```
novo_valor = 14
```

```
def change_value(var, novo_valor):
```

```
    print('Valor do 1º argumento: ',var,'\nValor do 2º argumento',novo_valor)
```

```
change_value(var,novo_valor)
```

```
Valor do 1º argumento: 7
```

```
Valor do 2º argumento 14
```


Funções – com 2 argumentos e com retorno

- Passagem de dois argumentos para a função e devolução do valor calculado

```
num1 = 6
num2 = 3

def division(num1,num2):
    result = num1/num2
    return result

print("A divisão dos valores é: ",division(num1,num2))
```

A divisão dos valores é: 2.0

Funções – outro exemplo

- Passagem de argumento para função e devolução de mensagem

```
def even_odd(num):  
    if num%2==0:  
        return f'O número {num} é par!'  
    else:  
        return f'O número {num} é ímpar!'  
  
even_odd(int(input("Insira um valor inteiro!")))
```

Insira um valor inteiro! 8

'O número 8 é par!'

Funções – com parâmetros default

- É possível uma função ter um ou mais parâmetros default, devem ser sempre os parâmetros do final

#Funções com parâmetros default

```
def calculate_BMI(weight, height, name):  
    bmi = weight/pow(height,2)  
    return "{}, seu IMC é {:.2f}.".format(name, bmi)
```

```
calculate_BMI(80,1.70,"John")
```

'John, seu IMC é 27.68.'

#Funções com parâmetros default

```
def calculate_BMI(weight, height, name="Prezado"):  
    bmi = weight/pow(height,2)  
    return "{}, seu IMC é {:.2f}.".format(name, bmi)
```

```
calculate_BMI(80,1.70)
```

'Prezado, seu IMC é 27.68.'

Funções – com parâmetros default

- Um exemplo com 2 parâmetros default

```
#Funções com parâmetros default  
  
def calculate_BMI(weight, height=1.5, name="Prezado"):  
    bmi = weight/pow(height,2)  
    return "{}, seu IMC é {:.2f}.".format(name, bmi)  
  
calculate_BMI(60)
```

'Prezado, seu IMC é 26.67.'

Funções – com parâmetros default

- Um exemplo com 3 parâmetros default, e passagem de 2 valores
- Nestas situações, deve sempre referir-se os parâmetros que estamos a enviar, como ilustra o exemplo

```
def configurar(som=True, brilho=50, modo="normal"):
    print(f"Som: {som}, Brilho: {brilho}, Modo: {modo}")

configurar(brilho=80, modo="noturno")
```

Funções – que recebem um número indefinido de parâmetros

- É possível escrever uma função que receba um número indefinido de parâmetros, recorrendo a `*args`

```
def print_all(*args):  
    for arg in args:  
        print(arg)
```

```
print_all(1, 2, 'example', 5)
```

1

2

example

5

Funções – que recebem um número indefinido de parâmetros e as suas chaves

- É possível escrever uma função que receba um número indefinido de parâmetros, bem como as suas chaves, recorrendo ao `**kwargs`

```
def print_data(**kwargs):  
    for chave, valor in kwargs.items():  
        print(f"{chave}: {valor}")  
  
print_data(name="Peter", age=25, city="Lisboa")
```

```
name: Peter  
age: 25  
city: Lisboa
```